



重慶理工大學

实验报告

课程 _____ 嵌入式系统及应用 _____

题目 _____ 串口操作的软件时钟日历实验 _____

院系名称 _____ 计算机科学与工程学院 _____

专业 _____ 物联网工程 _____ 班级 _____

学生姓名 _____ 学号 _____

任课教师 _____

时 间 _____

1 软件时钟日历中定时器设计

本工程使用 STM32F103RCT6 的 TIM2 作为软件时钟的基础定时源。程序启动后开启 TIM2 时钟，把 PSC 写为 7、ARR 写为 999。在默认 8MHz 时钟下，定时器计数频率先变为 1MHz，再由自动重装值分频为 1ms 产生一次更新事件。

主循环中调用 tim2_1ms_arrived() 读取 TIM2 的 UIF 标志位。若标志位已经置位，程序清除该标志，然后调用 NOW::tick_ms() 给当前时间加 1ms。这样定时工作由硬件定时器给出节拍，日期和时间的进位则在软件中完成。

结构/字段	主要成员	作用
DAT	year、month、day、week	保存年月日和星期，星期用 0~6 表示 SUN~SAT。
TIME	hour、minute、second、ms	保存时分秒与毫秒成员，满足实验要求中的毫秒显示。
NOW	dat、time	组合日期和时间，并负责毫秒、秒、分钟、小时和日期进位。

NOW::tick_ms() 每次只处理 1ms 进位；当 ms 到 1000 时清零并进入 add_second()。add_second() 依次处理 60 秒、60 分和 24 小时的进位。跨天时 add_day() 根据 month_days() 判断当月天数，同时更新 week。闰年由 is_leap_year() 判断，因此 2 月 28 日、2 月 29 日等情况能够按年份区分。

2 通讯协议设计

本实验使用 USART1 与 PC 通信，PA9 作为 USART1_TX，PA10 作为 USART1_RX。USART1 的 BRR 写入 0x45，对应 8MHz 外设时钟下约 115200 波特率。PC 端使用 USB 转 TTL 模块连接，测试端口为 /dev/cu.usbserial-310。

STM32 向 PC 发送当前时间时，采用固定文本格式：yyyy-mm-dd,WEEK,hh:mm:ss.ms，并在行尾加 \r\n。例如 2026-06-01,MON,12:34:56.789。固定宽度数字便于观察，也便于后续程序读取。

PC 修改当前时间时，采用以 > 开头、回车结束的命令格式：>yyyy-mm-dd hh:mm:ss.ms。程序接收到一整行后调用 parse_set_time() 解析，合法则更新 NOW 并回复 OK，不合法则回复 ERR。星期不由 PC 手动输入，而是由 calc_week() 根据日期重新计算。

方向	格式	示例	说明
STM32→PC	yyyy-mm-dd,WEEK,hh:mm:ss.ms	2026-05-31,SUN,00:00:41.300	每 100ms 主动发送一次当前 NOW。
PC→STM32	>yyyy-mm-dd hh:mm:ss.ms\r\n	>2026-06-01 12:34:56.789	修改当前 NOW，命令以回车或换行结束。
STM32→PC	OK / ERR	OK	OK 表示修改成功，ERR 表示格式或日期时间非法。

3 串口操作的软件数字钟设计实验

本工程完成了 NOW 的日期时间计算，并通过串口按指定格式发送到 PC、串口改时协议；SPI 显示和闹钟蜂鸣器没有完成。

3.1 程序结构设计

程序总体流程为：上电后获取外设访问权限，初始化 GPIOA、USART1 和 TIM2，然后创建 NOW、输入缓冲区和发送计数器。进入主循环后，程序先处理串口接收，再判断 TIM2 是否到达 1ms。若 1ms 到达，就更新时间并累计发送计数；计数达到 100ms 后通过 send_now() 发送一行当前时间。

步骤	处理内容	对应函数
1	使能 GPIOA、USART1、TIM2，并配置 PA9/PA10。	setup_clock_gpio_usart_timer()
2	TIM2 产生 1ms 节拍，主循环轮询 UIF 标志。	tim2_1ms_arrived()
3	根据 1ms 节拍推进 NOW。	NOW::tick_ms()、 add_second()、add_day()
4	周期性输出当前时间。	send_now()

5	接收 PC 命令并修改当前时间。	handle_serial_input()、 parse_set_time()
---	------------------	--

串口接收部分使用 input_buffer 保存普通字符，遇到 \r 或 \n 时认为一帧命令结束。缓冲区长度设置为 40 字节，足够保存一条 >yyyy-mm-dd hh:mm:ss.ms 命令。若输入过长，程序清空缓冲区并回反 ERR。

3.2 程序源代码分析

setup_clock_gpio_usart_timer() 负责硬件初始化。GPIOA 的 CRH 中，PA9 配置为复用推挽输出，PA10 配置为浮空输入；USART1 开启 UE、TE、RE；TIM2 清零控制寄存器后设置 PSC、ARR、CNT，再通过 EGR 产生一次更新，最后打开 CEN 开始计数。

Uart1::write_byte() 读取 SR 的 TXE 位，等待发送数据寄存器为空后写 DR。read_byte() 读取 RXNE 位，有数据时返回 DR 的低 8 位。

parse_set_time() 按固定位置读取年月日、时分秒和毫秒。read_number() 负责读取指定长度的十进制数字，expect_byte() 用来检查 -、:、. 等分隔符。check_datetime() 会检查月份、日期、时分秒和毫秒范围，因此，2026-02-30 或 25:00:00.000 这类非法输入会被拒绝。

calc_week() 以 2000-01-01（星期六）为基准，累加到目标日期前一天的天数，再对 7 取余得到星期。例如把时间修改为 2026-06-01 12:34:56.789 后，程序自动输出 MON。

3.3 实验结果及分析

工程使用 cargo build --release --offline 编译通过，再通过 cargo embed --release 下载到 STM32F103RCT6。串口端口为 /dev/cu.usbserial-310，波特率设置 115200。打开 Serail Monitor 后可以看到开发板持续输出软件时钟。

测试项目	输入/操作	观察结果	结果分析
上电运行	复位开发板	持续输出 2026-05-31,SUN,00:00:xx.xxx	NOW 初值正确，TIM2 1ms 节拍和串口发送工作正常。
毫秒显示	观察连续输出	毫秒按 100ms 间隔变化，如 .300、.400、.500	程序内部按 1ms 计时，显示周期为 100ms。

串口改时	>2026-06-01 12:34:56.789	返回 OK, 并输出 2026-06-01,MON,12:34:56.789	协议解析成功, NOW 被更新, 星期 自动重新计算。
错误命令	>2026-02-30 10:00:00.000	返回 ERR	日期非法, check_datetime() 拒 绝写入。

4 思考题

(1) 硬件定时与软件延时的区别?

软件延时通常通过循环或忙等待实现, 写法简单, 但 CPU 在延时期间基本不能做其它工作, 延时精度还会受 CPU 主频和中断等因素影响。硬件定时由定时器外设计数产生事件, CPU 只需要读取标志位或响应中断, 计时更稳定, 也更适合软件时钟、采样和周期发送等任务。

(2) 用定时器设计的时钟日历有那些误差? 有哪些方法可以校正这些误差?

误差主要来自时钟源本身的频率偏差、外界温度变化、分频参数、轮询处理延迟和串口发送占用时间。

校正方法包括: 使用精度更高的外部晶振或 RTC 时钟源; 把 TIM2 改为中断方式; 定期通过 PC 串口发送网络标准时间进行校准; 在软件中记录累计误差, 按固定周期补偿。